CodeGuide

Assistente de aprendizagem de JavaScript com inteligência artificial

---

DOCUMENTAÇÃO DO PROJETO

AUTORA
Camila Alvarenga

FORMAÇÃO ACADÊMICA
Sistemas de Informação
Centro Universitário de Volta Redonda (UniFOA)

Projeto independente
Novembro de 2026

Sumário

1. Visão geral **2**

2. Problema e contexto **3**

3. Público-alvo **3**

4. Proposta de valor **4**

5. Escopo do MVP **4**

6. Arquitetura e tecnologias **5**

7. Comportamento do assistente **6**

8. Testes realizados **7**

Teste 1: Identificação de uso inadequado do operador de atribuição em uma condição ..7

Teste 2: Identificação de inconsistência lógica 8

Teste 3: Reconhecimento de código correto 8

Teste 4: Orientação progressiva 9

Teste 5: Identificação de condição repetida 9

Evidências da interação 10

Análise dos resultados 11

9. Limitações conhecidas **11**

10. Estado atual do projeto **11**

1. Visão geral

O CodeGuide é um assistente de aprendizagem de JavaScript desenvolvido com o objetivo de auxiliar estudantes iniciantes na compreensão de códigos, identificação de possíveis erros e entendimento de conceitos fundamentais da linguagem.

A solução utiliza inteligência artificial para interpretar trechos de código e fornecer explicações em linguagem acessível, buscando tornar o processo de aprendizagem mais compreensível e incentivar a autonomia do estudante.

Além de apontar possíveis problemas, o assistente foi configurado para oferecer orientações progressivas, explicações sobre possíveis correções e sugestões de exercícios relacionados ao conteúdo apresentado.

O projeto foi desenvolvido utilizando **n8n** e integração com o modelo **Google Gemini 3.1 Flash-Lite**.

2. Problema e contexto

O aprendizado de programação envolve desafios que vão além da compreensão da sintaxe de uma linguagem. Para estudantes iniciantes, interpretar mensagens de erro, identificar inconsistências na lógica e compreender por que determinado código não apresenta o comportamento esperado pode ser uma dificuldade.

Muitas vezes, o estudante consegue reproduzir estruturas de código, mas encontra obstáculos ao tentar identificar sozinho a origem de um problema ou compreender o raciocínio necessário para solucioná-lo.

A partir dessa percepção, surgiu a proposta do CodeGuide: desenvolver um assistente que não apenas indique possíveis erros, mas também explique o que está acontecendo e ofereça orientações que estimulem o estudante a construir seu próprio entendimento.

Essa necessidade representa a **motivação inicial do projeto**, não uma conclusão obtida por meio de pesquisa estruturada com usuários.

3. Público-alvo

O público-alvo principal do CodeGuide são **estudantes iniciantes em JavaScript**, especialmente aqueles que estão tendo os primeiros contatos com programação e precisam de apoio para compreender estruturas, condições, operadores e lógica de código.

A ferramenta também pode ser utilizada por pessoas que estudam programação de forma independente e desejam obter explicações complementares durante a prática.

4. Proposta de valor

A proposta do CodeGuide é oferecer um apoio acessível ao aprendizado de JavaScript, utilizando inteligência artificial para tornar a análise de código mais compreensível para iniciantes.

Seus principais diferenciais são:

- **Explicações acessíveis:** apresenta conceitos e possíveis erros utilizando linguagem simples e adequada a estudantes iniciantes.
- **Análise de código:** interpreta trechos de JavaScript e identifica possíveis problemas de sintaxe ou lógica.
- **Orientação progressiva:** oferece pistas antes de apresentar uma solução completa, quando apropriado, incentivando o estudante a tentar resolver o problema.
- **Explicação das correções:** esclarece o motivo de uma possível correção e o conceito envolvido.
- **Exercícios complementares:** sugere atividades relacionadas ao conteúdo para estimular a prática.

A proposta não é substituir o processo de aprendizagem, mas oferecer suporte durante o desenvolvimento do raciocínio lógico e da compreensão da linguagem.

5. Escopo do MVP

A primeira versão do CodeGuide foi concebida para concentrar suas funcionalidades principais em uma interação direta entre estudante e assistente de inteligência artificial.

O escopo contempla:

Funcionalidade	Descrição
Recebimento de mensagens	Permite que o estudante envie perguntas e trechos de código JavaScript.
Análise de código	Interpreta o conteúdo enviado e identifica possíveis problemas.
Explicação de erros	Apresenta explicações sobre os problemas identificados.
Pistas progressivas	Oferece orientações para que o estudante tente encontrar a solução.
Explicação de correções	Explica possíveis ajustes e os conceitos envolvidos.
Exercícios complementares	Sugere atividades relacionadas ao assunto abordado.

Essas funcionalidades representam o **escopo proposto** para o MVP. Os testes manuais realizados permitiram observar parte desses comportamentos, mas **não representam uma validação completa** de todas as funcionalidades.

6. Arquitetura e tecnologias

O CodeGuide utiliza uma arquitetura baseada em automação e inteligência artificial, estruturada por meio do n8n.

O workflow atual é composto por três elementos principais:

1. When Chat Message Received

Responsável por receber a mensagem enviada pelo estudante e iniciar a interação no workflow.

2. AI Agent

Responsável por coordenar o processamento da mensagem e orientar a geração da resposta conforme as instruções configuradas no sistema.

3. Google Gemini Chat Model

Modelo de linguagem conectado ao AI Agent, responsável pela geração das respostas. O projeto utiliza o Google Gemini 3.1 Flash-Lite.

A integração permite que a mensagem recebida seja encaminhada ao agente, que utiliza o modelo de inteligência artificial para elaborar uma resposta de acordo com as instruções definidas.

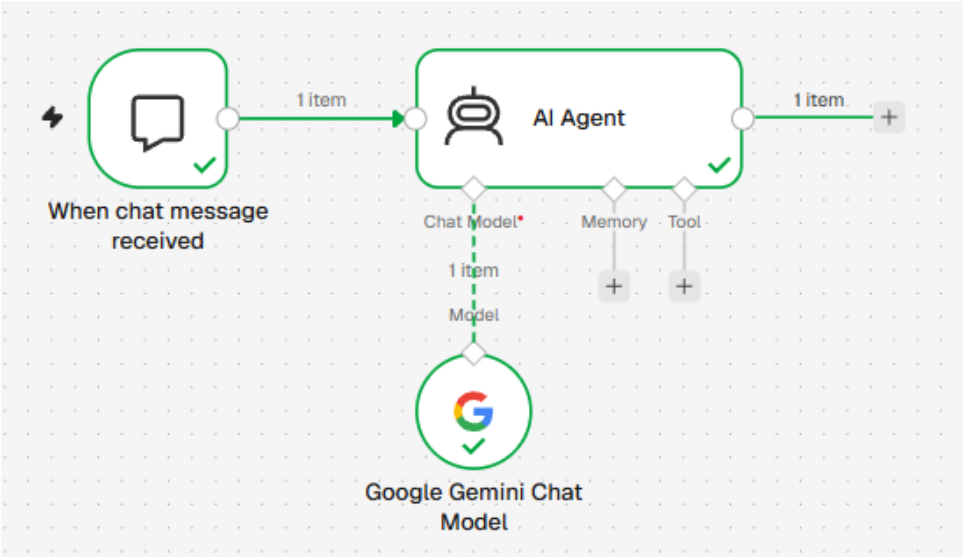


Figura 1 — Workflow do CodeGuide implementado no n8n, demonstrando a integração entre o recebimento de mensagens, o agente de IA e o modelo Gemini.

7. Comportamento do assistente

O comportamento do CodeGuide é orientado por instruções configuradas no campo **System Message** do AI Agent.

Essas instruções definem como o assistente deve interpretar as mensagens recebidas e estruturar suas respostas, considerando o público iniciante ao qual a ferramenta se destina.

Entre as orientações estabelecidas estão:

- Analisar trechos de código JavaScript e mensagens de erro enviados pelo estudante.
- Identificar possíveis problemas de sintaxe ou lógica.
- Explicar os problemas utilizando linguagem simples e acessível.
- Oferecer pistas progressivas antes de apresentar uma solução completa, quando apropriado.
- Explicar possíveis correções e os motivos por trás delas.
- Sugerir exercícios relacionados ao conteúdo abordado.
- Evitar inventar erros quando não identificar problemas.
- Solicitar mais contexto quando as informações fornecidas forem insuficientes.
- Informar que não executa o código, mas realiza uma análise interpretativa do conteúdo enviado.

Dessa forma, o assistente foi configurado para atuar como um apoio ao aprendizado, priorizando a compreensão dos conceitos e o desenvolvimento do raciocínio do estudante.

Como utiliza um modelo de linguagem, suas respostas **podem apresentar imprecisões** e devem ser interpretadas como orientações, não como garantia de correção.

8. Testes realizados

Para observar o comportamento do CodeGuide, foram realizados **cinco testes manuais** com diferentes situações envolvendo códigos JavaScript.

Os testes buscaram verificar como o assistente se comporta diante de erros de sintaxe ou lógica, códigos corretos e situações que exigem orientação progressiva.

Teste 1: Identificação de uso inadequado do operador de atribuição em uma condição

```
let idade = 15;

if (idade = 18) {
  console.log("Maior de idade");
}
```

O assistente identificou o uso do operador de atribuição (=) dentro da condição, explicando sua diferença em relação aos operadores de comparação. Também apresentou uma explicação sobre o funcionamento desse comportamento em JavaScript.

Durante esse teste, observou-se que a explicação foi apresentada corretamente em relação ao conceito, embora a resposta tenha revelado a solução mais rapidamente do que a proposta de orientação progressiva pretendia.

Teste 2: Identificação de inconsistência lógica

```
let temperatura = 30;

if (temperatura < 20) {
  console.log("Está quente!");
} else {
  console.log("Está frio!");
}
```

O assistente identificou uma inconsistência entre a condição estabelecida e as mensagens exibidas pelos blocos condicionais.

A resposta explicou que o comportamento esperado não correspondia à lógica apresentada e ofereceu uma orientação para que o estudante pudesse identificar o problema.

Teste 3: Reconhecimento de código correto

```
let nota = 8;

if (nota >= 7) {
  console.log("Aprovado");
} else {
  console.log("Reprovado");
}
```

Nesse cenário, o assistente reconheceu que a estrutura condicional estava correta e explicou como a comparação determinava qual bloco seria executado.

Também sugeriu um exercício complementar envolvendo a utilização de `else if`, ampliando a possibilidade de prática do conteúdo.

Teste 4: Orientação progressiva

```
let nota = 6;

if (nota >= 7) {
  console.log("Aprovado");
} else {
  console.log("Reprovado");
}
```

O estudante apresentou um código funcional e solicitou orientação para ampliar sua lógica, incluindo uma nova situação de resultado intermediário.

O assistente sugeriu a utilização de `else if` e ofereceu uma pista sobre onde inserir a nova condição, sem apresentar imediatamente o código completo.

Esse teste permitiu observar o comportamento de orientação progressiva proposto para o projeto.

Teste 5: Identificação de condição repetida

```
let nota = 6;

if (nota >= 7) {
  console.log("Aprovado");
} else if (nota >= 7) {
  console.log("Recuperação");
} else {
  console.log("Reprovado");
}
```

O assistente identificou que a segunda condição repetia a primeira e explicou por que o segundo bloco não seria alcançado quando a primeira condição fosse falsa.

Em seguida, apresentou uma pista para que o estudante identificasse qual faixa de notas deveria ser considerada na condição intermediária.

Evidências da interação

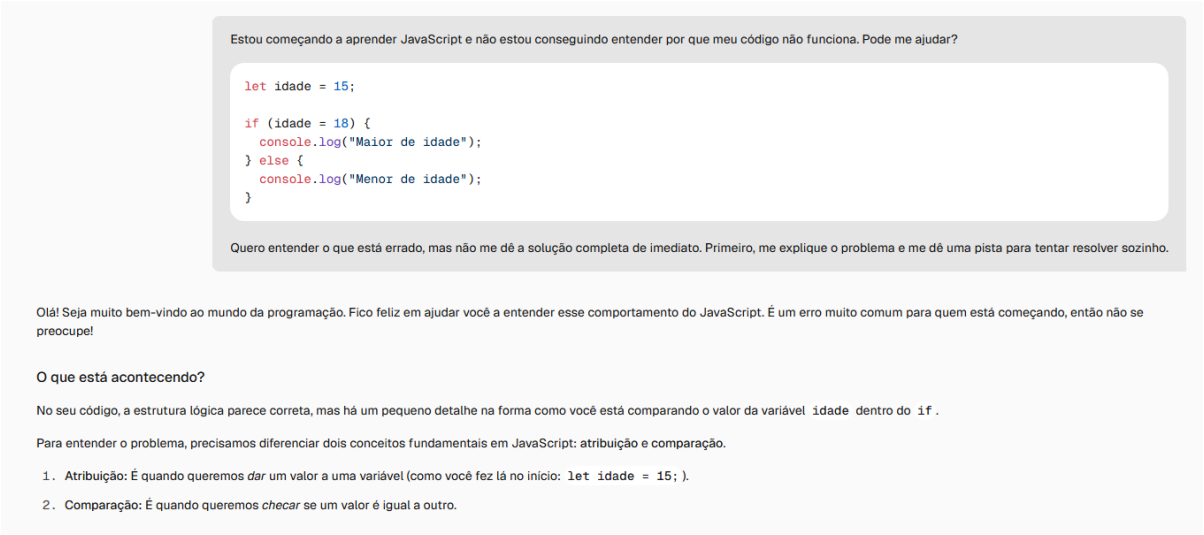
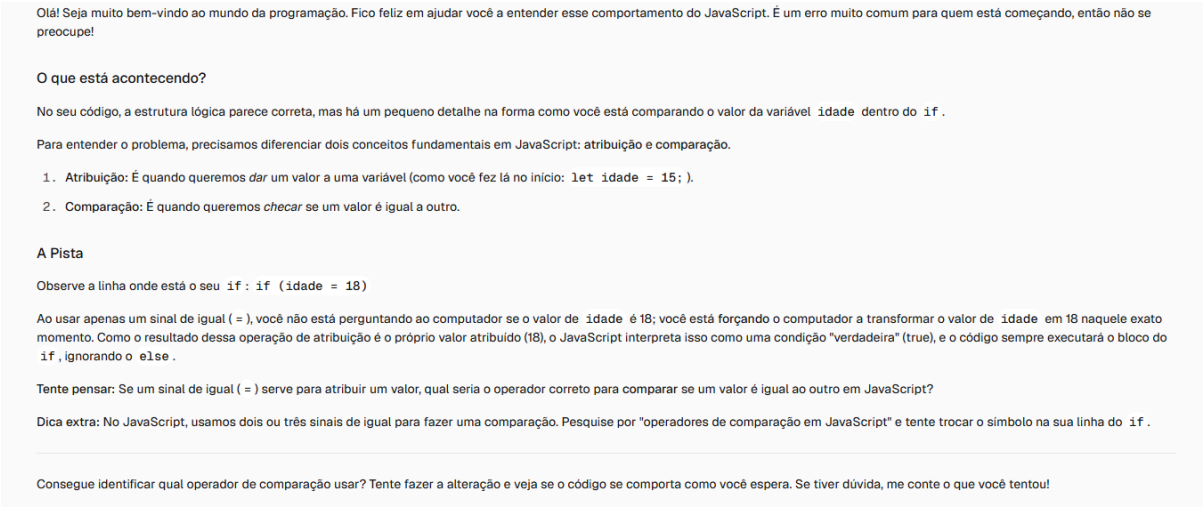


Figura 2 — Interação com o CodeGuide durante a identificação de um possível erro em uma condição JavaScript.


Olá! Seja muito bem-vindo ao mundo da programação. Fico feliz em ajudar você a entender esse comportamento do JavaScript. É um erro muito comum para quem está começando, então não se preocupe!

O que está acontecendo?

No seu código, a estrutura lógica parece correta, mas há um pequeno detalhe na forma como você está comparando o valor da variável `idade` dentro do `if`.

Para entender o problema, precisamos diferenciar dois conceitos fundamentais em JavaScript: atribuição e comparação.

1. Atribuição: É quando queremos *dar* um valor a uma variável (como você fez lá no início: `let idade = 15;`).
2. Comparação: É quando queremos *checar* se um valor é igual a outro.

A Pista

Observe a linha onde está o seu `if`: `if (idade = 18)`

Ao usar apenas um sinal de igual (=), você não está perguntando ao computador se o valor de `idade` é 18; você está forçando o computador a transformar o valor de `idade` em 18 naquele exato momento. Como o resultado dessa operação de atribuição é o próprio valor atribuído (18), o JavaScript interpreta isso como uma condição "verdadeira" (true), e o código sempre executará o bloco do `if`, ignorando o `else`.

Tente pensar: Se um sinal de igual (=) serve para atribuir um valor, qual seria o operador correto para comparar se um valor é igual ao outro em JavaScript?

Dica extra: No JavaScript, usamos dois ou três sinais de igual para fazer uma comparação. Pesquise por "operadores de comparação em JavaScript" e tente trocar o símbolo na sua linha do `if`.

Consegue identificar qual operador de comparação usar? Tente fazer a alteração e veja se o código se comporta como você espera. Se tiver dúvida, me conte o que você tentou!

Figura 3 — Continuação da interação, demonstrando a utilização de pistas para estimular o raciocínio do estudante.

Análise dos resultados

Os cinco testes permitiram observar diferentes comportamentos do CodeGuide diante de situações comuns no aprendizado de JavaScript.

O assistente conseguiu identificar possíveis inconsistências lógicas, explicar o uso inadequado de operadores, reconhecer um exemplo de código correto e oferecer orientações progressivas em situações que exigiam maior reflexão.

Também foi possível observar que o comportamento nem sempre segue integralmente a proposta de apresentar pistas antes da solução, indicando que as respostas podem variar conforme a situação apresentada.

> Os resultados representam observações obtidas em testes manuais específicos e não comprovam, por si só, a eficácia da ferramenta no processo de aprendizagem.

9. Limitações conhecidas

O CodeGuide utiliza um modelo de linguagem para interpretar os códigos enviados pelos estudantes. Por esse motivo, suas respostas podem apresentar imprecisões, interpretações incorretas ou sugestões de correção inadequadas.

Atualmente, o sistema não executa os códigos enviados nem verifica seu funcionamento em um ambiente de execução. Sua análise é baseada na interpretação do conteúdo fornecido pelo estudante.

A avaliação realizada até o momento também possui um escopo inicial, composto por cinco cenários de testes manuais, sem uma avaliação estruturada com usuários reais.

Essas limitações são consideradas no estágio atual do projeto e indicam aspectos que podem ser explorados em versões futuras.

10. Estado atual do projeto

O CodeGuide possui uma primeira versão funcional implementada no n8n e integrada ao modelo Google Gemini 3.1 Flash-Lite.

O workflow permite receber mensagens dos estudantes, encaminhá-las ao agente de inteligência artificial e gerar respostas orientadas pelas instruções definidas no System Message.

A configuração atual permite que o assistente interprete códigos JavaScript, identifique possíveis problemas, apresente explicações acessíveis e utilize pistas progressivas para estimular o raciocínio do estudante.

Foram realizados cinco testes manuais, nos quais foi possível observar diferentes comportamentos do assistente, incluindo a identificação de problemas em códigos, o reconhecimento de exemplos corretos e a orientação por meio de pistas.

Assim, o projeto conta com uma primeira versão funcional, com fluxo de funcionamento implementado, comportamento configurado e testes iniciais documentados.